SkiPrepCoach — Core Engine Specification v0.1

1. Purpose

SkiPrepCoach is a server-side decision engine that answers one question:

> What is the best next action for this user right now?

The client is thin. It displays the recommended action, collects the result, and sends that result back to the server.

The engine optimizes for safe, steady progress toward skiing capability by using:

- user goals
- exercise metadata
- recovery rules
- capability targets
- recent training history
- pain and effort feedback
- estimated readiness
- estimated warm-up state
- variation rules

There are no fixed workouts, snack mode, or user-selected plans. Everything is expressed as a repeated loop:

```
GET /next
→ user performs action
→ POST /result
→ engine updates state
→ GET /next
```

---

2. Core Data Objects

2.1 User Profile

Stores stable user-level information.

```
{
  "userId": "user-001",
```

```
  "displayName": "Will",
  "primaryGoal": "ski_resilience",
  "targetDate": "2026-11-01",
  "availableEquipment": [
    "gym",
    "dumbbells",
    "barbell",
    "bike",
    "rollerblades",
    "pickleball_court",
    "hiking_trails"
  ],
  "constraints": {
    "kneeSensitivity": true,
    "lowBackCaution": true,
    "avoidBulking": true
  },
  "preferences": {
    "likes": ["rollerblading", "hiking", "pickleball"],
    "dislikes": [],
    "preferredSessionStyle": "next_action"
  }
}
```

Used by `next` logic to filter exercises, prioritize goals, and bias recommendations toward enjoyable options.

2.2 Capability Definitions

Capabilities are things the engine tries to improve.

```
{
  "capabilities": {
    "knee_capacity": {
      "name": "Knee Capacity",
      "icon": "🦿",
      "priority": 10,
      "description": "Ability of knees and surrounding tissues to tolerate
skiing-relevant load."
    },
    "lower_body_strength": {
      "name": "Lower Body Strength",
      "icon": "🦵",
      "priority": 9
    },
```

```
    "posterior_chain": {
      "name": "Posterior Chain",
      "icon": "🍑",
      "priority": 8
    },
    "balance": {
      "name": "Balance",
      "icon": "🫧",
      "priority": 9
    },
    "mobility": {
      "name": "Mobility",
      "icon": "🧘",
      "priority": 7
    },
    "aerobic_endurance": {
      "name": "Aerobic Endurance",
      "icon": "🐢",
      "priority": 7
    },
    "stamina": {
      "name": "Stamina",
      "icon": "🔥",
      "priority": 8
    },
    "reaction": {
      "name": "Reaction",
      "icon": "⚡",
      "priority": 6
    },
    "upper_body_strength": {
      "name": "Upper Body Strength",
      "icon": "💪",
      "priority": 5
    },
    "fall_resilience": {
      "name": "Fall Resilience",
      "icon": "🛡️",
      "priority": 6
    }
  }
}
```


Used to score candidate actions. Higher-priority capabilities receive more weight, especially if currently undertrained or limiting.

2.3 User Capability State

Stores the current estimate of user ability.

```
{
  "userId": "user-001",
  "capabilities": {
    "knee_capacity": {
      "score": 22,
      "trend": "improving",
      "lastTrainedAt": "2026-07-04T09:20:00-07:00",
      "fatigue": 12
    },
    "lower_body_strength": {
      "score": 18,
      "trend": "stable",
      "lastTrainedAt": "2026-07-03T16:00:00-07:00",
      "fatigue": 30
    },
    "balance": {
      "score": 28,
      "trend": "improving",
      "lastTrainedAt": "2026-07-04T11:30:00-07:00",
      "fatigue": 4
    }
  }
}
```

Fields:

- `score`: rough capability level, not visible as a medical metric.
- `trend`: improving, stable, declining, unknown.
- `lastTrainedAt`: used for recovery and variation.
- `fatigue`: estimated current fatigue cost for that capability.

2.4 Exercise Definition

Base exercise information. This should be compatible with Free Exercise DB style fields, then extended.

```
{
  "id": "romanian_deadlift_barbell",
  "name": "Barbell Romanian Deadlift",
  "icon": "🏋",
```

```
  "baseSource": "free-exercise-db",
  "movementPattern": "hinge",
  "familyId": "hip_hinge",
  "variantTags": ["barbell", "bilateral", "posterior_chain", "strength"],
  "equipment": ["barbell", "plates"],
  "bodyParts": ["hamstrings", "glutes", "back"],
  "instructions": [
    "Stand tall holding the barbell in front of your thighs.",
    "Hinge at the hips while keeping your back neutral.",
    "Lower until you feel a hamstring stretch.",
    "Return to standing by driving the hips forward."
  ],
  "safetyNotes": [
    "Do not round the lower back.",
    "Do not perform cold.",
    "Stop if back pain or sharp knee pain occurs."
  ],
  "requiresWarmth": "warm",
  "riskLevel": "moderate",
  "recoveryClass": "heavy_strength",
  "snackSafeWhenCold": false,
  "capabilityEffects": {
    "posterior_chain": 8,
    "lower_body_strength": 5,
    "fall_resilience": 2
  },
  "fatigueCost": {
    "posterior_chain": 20,
    "lower_body_strength": 12,
    "low_back": 12,
    "knee_capacity": 4
  },
  "substitutes": [
    "romanian_deadlift_dumbbell",
    "kettlebell_deadlift",
    "hip_hinge_dowel"
  ],
  "regressions": [
    "hip_hinge_dowel",
    "romanian_deadlift_dumbbell_light"
  ],
  "progressions": [
    "romanian_deadlift_barbell_heavy",
    "single_leg_rdl"
  ]
}
```

Used by `next` logic for filtering, scoring, safety, recovery, variation, and explanation.

2.5 Exercise Prescription

A concrete recommended dose.

```
{
  "exerciseId": "wall_sit",
  "load": "bodyweight",
  "sets": 3,
  "durationSec": 30,
  "restSec": 60,
  "targetRpe": 5,
  "painLimit": 3,
  "estimatedDurationSec": 210
}
```

Rep-based example:

```
{
  "exerciseId": "bodyweight_squat",
  "load": "bodyweight",
  "sets": 2,
  "reps": 10,
  "tempo": {
    "downSec": 3,
    "pauseSec": 1,
    "upSec": 2
  },
  "restSec": 60,
  "targetRpe": 4,
  "painLimit": 3,
  "estimatedDurationSec": 180
}
```

Used as the returned `nextAction`.

2.6 Recovery Classes

Defines minimum recovery rules.

```
{
  "recoveryClasses": {
    "daily": {
      "minRestHours": 6,
      "maxPerDay": 6,
      "maxPerWeek": 42
    },
    "light": {
      "minRestHours": 12,
      "maxPerDay": 3,
      "maxPerWeek": 14
    },
    "moderate": {
      "minRestHours": 24,
      "maxPerDay": 2,
      "maxPerWeek": 6
    },
    "heavy_strength": {
      "minRestHours": 48,
      "maxPerDay": 1,
      "maxPerWeek": 3
    },
    "max_strength": {
      "minRestHours": 72,
      "maxPerDay": 1,
      "maxPerWeek": 2
    },
    "plyometric": {
      "minRestHours": 48,
      "maxPerDay": 1,
      "maxPerWeek": 3
    },
    "hiit": {
      "minRestHours": 48,
      "maxPerDay": 1,
      "maxPerWeek": 2
    }
  }
}
```

Used to block actions that are not recovered.

2.7 User Activity History

Every completed or attempted action is stored as an event.

```
{
  "events": [
    {
      "eventId": "evt-001",
      "userId": "user-001",
      "type": "exercise_result",
      "startedAt": "2026-07-04T09:00:00-07:00",
      "completedAt": "2026-07-04T09:04:00-07:00",
      "exerciseId": "wall_sit",
      "prescribed": {
        "sets": 3,
        "durationSec": 30,
        "restSec": 60
      },
      "actual": {
        "setsCompleted": 3,
        "durationSecCompleted": 90,
        "load": "bodyweight",
        "maxPain": 1,
        "rpe": 5,
        "difficulty": "normal",
        "notes": "Felt fine."
      }
    }
  ]
}
```


Used to update capability, fatigue, warmth, exercise variation, readiness, and progression.

2.8 Readiness State

Computed from recent reports and optional biometric/manual data.


```
{
  "date": "2026-07-04",
  "painNow": 1,
  "morningStiffness": "none",
  "swelling": false,
  "stairs": "easy",
  "sleepQuality": "good",
  "restingHr": 52,
  "hrvStatus": "balanced",
  "bodyBattery": 78,
  "trainingReadiness": 82,
```

```
  "computedStatus": "green"
}
```

Rules:

- Red if swelling, limp, or pain ≥4.
- Yellow if pain 2–3, poor sleep, elevated fatigue, or bad recovery markers.
- Green if low pain, no swelling, normal movement, and acceptable fatigue.

Used early in the `next` pipeline.

2.9 Warmth State

Warmth is computed, not manually selected.

```
{
  "warmth": {
    "score": 42,
    "state": "warm",
    "updatedAt": "2026-07-04T09:04:00-07:00",
    "source": "computed"
  }
}
```

Suggested states:

```
{
  "cold": "0-19",
  "slightly_warm": "20-39",
  "warm": "40-69",
  "very_warm": "70-89",
  "fatigued": "90+ with high fatigue"
}
```

Warmth increases after movement and decays with inactivity.

Example warmth effects:

```
{
  "walk_5_min": 10,
  "mobility_light": 8,
  "wall_sit": 8,
```

```
  "bodyweight_squat": 12,
  "heavy_strength": 25,
  "rollerblade_20_min": 35
}
```

Used to block cold-unsafe exercises and sequence actions naturally.

---

3. Server API

3.1 Get Next Action

```
GET /api/users/{userId}/next
```

Returns:

```
{
  "nextAction": {
    "type": "exercise",
    "exerciseId": "wall_sit",
    "title": "Wall Sit",
    "icon": "🪑",
    "prescription": {
      "sets": 3,
      "durationSec": 30,
      "restSec": 60,
      "targetRpe": 5,
      "painLimit": 3
    },
    "estimatedDurationSec": 210,
    "instructions": [
      "Lean against a wall with feet hip-width apart.",
      "Slide down only as far as comfortable.",
      "Keep knees aligned with feet.",
      "Stop if pain exceeds 3/10."
    ],
    "completionQuestions": [
      "How many sets did you complete?",
      "What was the highest pain level?",
      "What was the effort level, 1-10?",
      "Anything unusual?"
    ],
    "why": [
      "Knee capacity is currently a high-priority capability.",
```

```
"Wall sits provide useful tendon and quadriceps stimulus with low movement
stress.",
      "You are not warm enough for heavier lower-body strength work yet."
    ]
  },
  "todayProgress": {
    "status": "in_progress",
    "stimulusScore": 38,
    "targetStimulusScore": 70,
    "percentComplete": 54
  },
  "stateSummary": {
    "readiness": "green",
    "warmth": "slightly_warm",
    "limitingCapabilities": ["knee_capacity", "lower_body_strength"]
  }
}
```

If rest is best:

```
{
  "nextAction": {
    "type": "rest",
    "title": "Rest Is the Best Next Action",
    "estimatedDurationSec": null,
    "instructions": [
      "No more training is recommended right now.",
      "Normal walking and daily activity are fine.",
      "Resume when the app recommends another action."
    ],
    "completionQuestions": [],
    "why": [
      "You have already reached today's useful training stimulus.",
      "Additional lower-body work would create more fatigue than adaptation.",
      "Recovery now increases the chance of productive training tomorrow."
    ]
  },
  "todayProgress": {
    "status": "complete",
    "stimulusScore": 76,
    "targetStimulusScore": 70,
    "percentComplete": 100
  }
}
```

3.2 Submit Result

```
POST /api/users/{userId}/results
```

Body:

```
{
  "recommendationId": "rec-20260704-001",
  "exerciseId": "wall_sit",
  "startedAt": "2026-07-04T09:00:00-07:00",
  "completedAt": "2026-07-04T09:04:00-07:00",
  "actual": {
    "setsCompleted": 3,
    "durationSecCompleted": 90,
    "maxPain": 1,
    "rpe": 5,
    "difficulty": "normal",
    "notes": "Felt good."
  }
}
```

Server responsibilities:

1. Store event.
2. Update warmth.
3. Update fatigue.
4. Update capability estimates.
5. Update daily progress.
6. Update recovery predictions.
7. Make next recommendation available.

---

4. Next Decision Pipeline

The `next` engine must run in this order.

Step 1 — Load State

Load:

- user profile
- current capability state

- exercise library
- recovery classes
- recent events
- readiness state
- current date/time
- goal and target date

Step 2 — Update Derived State

Before choosing an action, recompute:

- current warmth
- decayed fatigue
- recovered capabilities
- today's stimulus
- readiness status
- recently repeated movement patterns
- blocked exercises
- limiting capabilities

This ensures stale state does not drive decisions.

Step 3 — Safety Veto

Immediately return rest/recovery if:

- pain now ≥ 4
- swelling reported
- limp or instability reported
- severe next-morning pain response
- red readiness state
- unsafe fatigue accumulation

Safety has veto power over all performance goals.

Possible output:

```
{
  "type": "rest",
  "reasonCodes": ["safety_red_day", "pain_too_high"]
}
```

Step 4 — Determine Whether Enough Has Been Done Today

Compute `todayStimulusScore`.

Example:

```
{
  "todayStimulusScore": 76,
  "targetStimulusScore": 70,
  "capabilityStimulus": {
    "knee_capacity": 22,
    "balance": 8,
    "mobility": 12,
    "aerobic_endurance": 15
  }
}
```

If enough useful stimulus has been achieved and no low-fatigue beneficial action remains, return rest.

This prevents overtraining.

Step 5 — Identify Limiting Capabilities

Score each capability.

Factors:

- low score relative to goal
- high goal priority
- undertrained recently
- gate requirement nearby
- trend declining
- relevant to current season

Example:

```
{
  "limitingCapabilities": [
    {
      "capabilityId": "knee_capacity",
      "score": 22,
      "priorityWeight": 10,
      "reason": "low_score_high_priority"
```

```
    },
    {
      "capabilityId": "lower_body_strength",
      "score": 18,
      "priorityWeight": 9,
      "reason": "low_score_high_priority"
    }
  ]
}
```

Step 6 — Generate Candidate Actions

Generate candidates from the exercise library.

An exercise is initially eligible if:

- user has equipment
- not medically constrained
- current level allows it
- readiness allows it
- warmth allows it
- recovery class allows it
- target capability is useful
- fatigue cost is acceptable
- pain history is acceptable

Example blocked exercise:

```
{
  "exerciseId": "romanian_deadlift_barbell",
  "blocked": true,
  "reasonCodes": ["not_warm_enough"]
}
```

Step 7 — Score Candidate Actions

Each candidate gets a score.

Suggested scoring factors:

```
score =
  capability benefit
+ goal priority
+ limiting capability bonus
+ recovery compatibility
+ warm-up compatibility
+ variation bonus
+ enjoyment bonus
+ gate preparation bonus
- fatigue cost
- pain risk
- recent repetition penalty
- complexity penalty
```

Example:

```
{
  "exerciseId": "wall_sit",
  "score": 82,
  "reasonCodes": [
    "trains_limiting_capability",
    "safe_when_slightly_warm",
    "low_fatigue_cost"
  ]
}
```

Step 8 — Apply Variation Rules

The engine should prefer useful variation, not random variation.

Hierarchy:

```
Capability → Movement Pattern → Exercise Family → Variant
```

Rules:

- Do not repeat the same exercise too often if safe substitutes exist.
- Do not vary so much that progression becomes unmeasurable.
- Prefer variants in the same movement family when the same training effect is desired.
- Use regressions if readiness or warmth is low.
- Use progressions only when recent results justify it.

Example:

```
{
  "movementPattern": "hinge",
  "recentVariants": ["romanian_deadlift_barbell"],
  "suggestedVariant": "romanian_deadlift_dumbbell",
  "reason": "same pattern, less repetition, appropriate load"
}
```

Step 9 — Select Dose

After selecting the exercise, choose dose.

Dose depends on:

- recent performance
- target RPE
- pain response
- recovery state
- warmth
- level
- goal for the action

Example:

```
{
  "exerciseId": "wall_sit",
  "doseReason": "increase_duration_slightly",
  "previous": {
    "sets": 3,
    "durationSec": 20,
    "maxPain": 1,
    "rpe": 4
  },
  "next": {
    "sets": 3,
    "durationSec": 30,
    "targetRpe": 5
  }
}
```

Progression should be conservative.

Step 10 — Build Explanation

Every recommendation must include:

1. What to do.
2. How to do it.
3. What to report afterward.
4. Why this was selected.
5. How it advances goals.
6. Why harder options were not selected, if relevant.

Use deterministic reason codes and message templates.

---

5. Result Processing Logic

When a result is submitted:

5.1 Store Event

Append the event to user history.

5.2 Update Warmth

Increase warmth based on completed work.

Then apply decay based on time.

5.3 Update Fatigue

Add fatigue according to exercise fatigue model and actual dose.

Example:

```
{
  "posterior_chain": "+12",
  "knee_capacity": "+4",
  "lower_body_strength": "+7"
}
```

5.4 Update Capability Scores

If completed successfully with acceptable pain and RPE, increment capability.

Example:

```
{
  "knee_capacity": "+0.2",
  "balance": "+0.1"
}
```

Do not increase capability if:

- pain exceeded limit
- form was poor
- user stopped early due to discomfort
- RPE was far above target

5.5 Update Pain Risk

If exercise caused pain or next-day soreness, reduce future score or require regression.

5.6 Update Variation History

Record movement pattern, family, and variant.

5.7 Update Today Progress

Increase daily stimulus score.

---

6. Daily Progress

The user should see progress for the day, but not a rigid plan.

```
{
  "date": "2026-07-04",
  "targetStimulusScore": 70,
  "currentStimulusScore": 38,
  "percentComplete": 54,
  "capabilityStimulus": {
    "knee_capacity": 16,
    "mobility": 8,
    "balance": 10,
    "aerobic_endurance": 4
  },
```

```
  "status": "in_progress"
}
```

This is used only to answer:

- Have we done enough today?
- Which capabilities have received enough stimulus?
- Which useful low-risk opportunities remain?

---

7. Initial MVP Exercise Set

Start with a small curated subset.

Examples:

- wall_sit
- bodyweight_squat
- step_up_low
- calf_raise
- single_leg_balance
- ankle_rocker
- hip_flexor_stretch
- ninety_ninety
- dead_bug
- pushup
- farmer_carry
- goblet_squat
- romanian_deadlift_dumbbell
- bike_easy
- walk_easy
- rollerblade_easy

More exercises can be imported from Free Exercise DB, but each needs SkiPrepCoach metadata before recommendation.

---

8. MVP Development Order

1. Define exercise JSON.
2. Define user state JSON.
3. Implement event storage.
4. Implement derived state update.
5. Implement safety veto.
6. Implement candidate generation.

7. Implement candidate scoring.
8. Implement `GET /next`.
9. Implement `POST /result`.
10. Add explanations from reason codes.
11. Add variation rules.
12. Add capability scoring.

---

9. Core Principle

SkiPrepCoach is not a workout calendar.

It is a closed-loop coaching engine.

Each recommendation is based on the current state of the athlete, and each completed action changes that state.

The app’s intelligence comes from the quality of:

- the exercise metadata
- the recovery model
- the capability model
- the user history
- the `next` decision logic

The UI exists only to show the next action and collect the result.